



Profesor:

Alonso Pérez Víctor Ernesto

Materia:

Fundamentos de Inteligencia Artificial

Proyecto:

Realidad Aumentada

Alumno:

Reyes Cruz Roberto Misael

## Tabla de contenido

|                             |    |
|-----------------------------|----|
| Introducción: .....         | 1  |
| Marco Teórico: .....        | 2  |
| Desarrollo:.....            | 2  |
| Código .....                | 7  |
| Explicación .....           | 16 |
| Resultados:.....            | 19 |
| Conclusión:.....            | 20 |
| Fuentes Bibliograficas..... | 20 |

## Introducción:

En este proyecto, exploramos la implementación de Realidad Aumentada (RA) utilizando Python y OpenCV. La RA es una tecnología que superpone información generada por computadora en el entorno real del usuario, creando una visión compuesta en tiempo real. Nuestro objetivo es detectar una imagen específica (superficie objetivo) y superponer un modelo 3D sobre ella, lo cual se logra mediante la detección de características y la correspondencia de puntos clave entre la superficie objetivo y la escena capturada por la cámara.

## Marco Teórico:

- **Realidad Aumentada (RA):** Combina elementos del mundo real con elementos virtuales para crear una realidad mixta.
- **OpenCV (Open Source Computer Vision Library):** Biblioteca de software de visión por computadora y aprendizaje automático.
- **Python:** Lenguaje de programación de alto nivel, interpretado y de propósito general.
- **ORB (Oriented FAST and Rotated BRIEF):** Algoritmo para la detección de características y la descripción de puntos clave en imágenes.
- **Homografía:** Transformación que mapea los puntos de una imagen a otra, manteniendo la perspectiva.
- **Modelos 3D y Archivos OBJ:** Formatos de archivo utilizados para representar modelos tridimensionales.

## Desarrollo:

La Realidad Aumentada (RA) es un campo interdisciplinario que implica la superposición de información generada por computadora en el mundo real, creando una realidad compuesta. Este proceso mejora la percepción del usuario del mundo real con información digital adicional. A continuación, se describen los temas clave de la RA relacionados con los códigos proporcionados:

### 1. Captura y Análisis de Imágenes en Tiempo Real

La RA depende de la capacidad de capturar y analizar el entorno del usuario en tiempo real. En el código proporcionado, esto se logra mediante la captura de video utilizando la cámara web. Cada fotograma capturado es procesado para detectar puntos clave que serán usados para entender la geometría y el contenido del entorno.

### 2. Detección de Puntos Clave y Descriptores

Los puntos clave son características distintivas en una imagen, como esquinas o áreas de textura única. En el código, se utiliza el algoritmo ORB para detectar y describir estos puntos clave. Los descriptores de ORB son representaciones binarias eficientes que describen los puntos clave y permiten su rápida comparación.

### 3. Coincidencia de Características y Homografía

Una vez detectados los puntos clave y calculados sus descriptores, se buscan coincidencias entre los puntos clave de la imagen objetivo y los puntos clave del fotograma capturado en tiempo real. Usando un algoritmo de fuerza bruta y la distancia de Hamming, se emparejan las características similares. Con suficientes coincidencias, se calcula una homografía, que es una transformación proyectiva que relaciona dos planos en el espacio.

### 4. Proyección 3D y Renderización

La homografía se utiliza junto con los parámetros intrínsecos de la cámara para calcular una matriz de proyección 3D. Esta matriz permite renderizar un modelo 3D en el contexto del entorno capturado, respetando la perspectiva y la escala correctas. La renderización implica dibujar el modelo 3D en el lugar correcto del fotograma de video, creando la ilusión de que el objeto virtual existe en el mundo real.

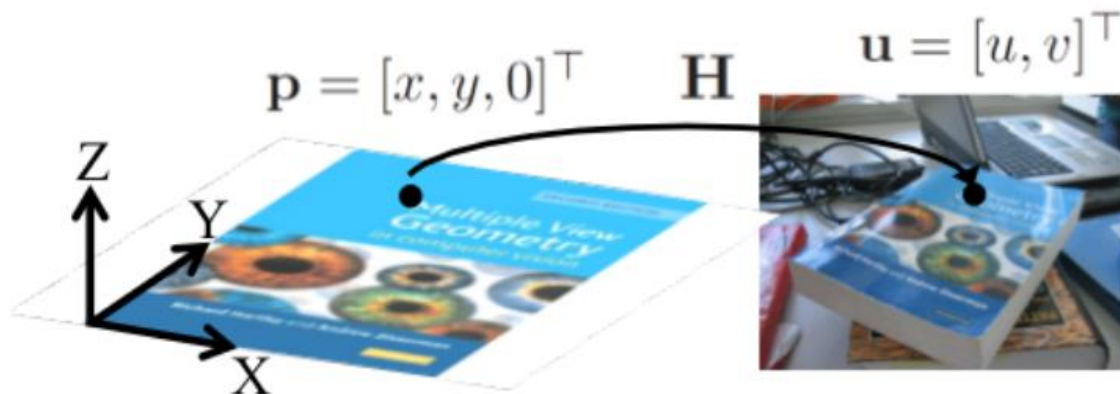
## 5. Interacción y Visualización en Tiempo Real

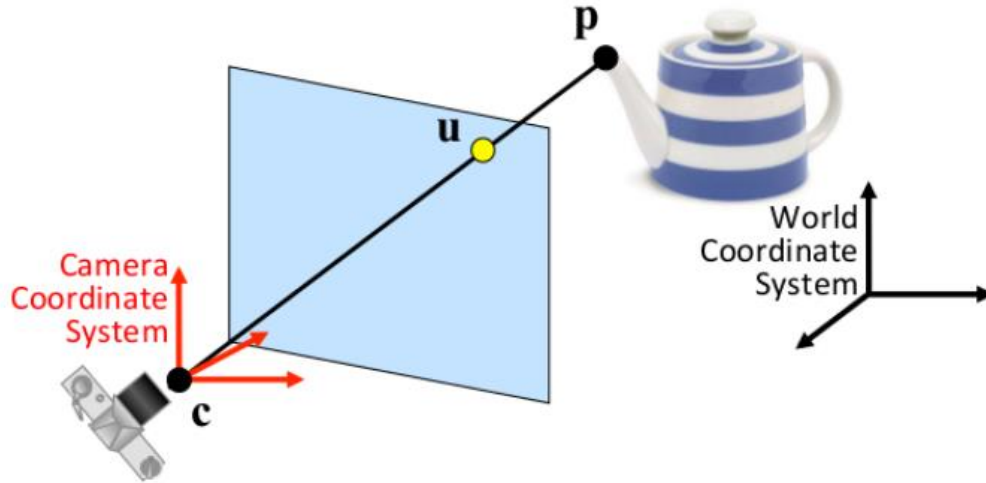
Finalmente, el sistema debe permitir la interacción del usuario con la RA y mostrar los resultados en tiempo real. El código proporcionado gestiona la visualización continua del video con el modelo 3D superpuesto y proporciona mecanismos para finalizar la sesión de RA.

### Estimación de homografía

Una vez que hemos identificado la superficie de referencia en el cuadro actual y tenemos un conjunto de coincidencias válidas podemos proceder a estimar la homografía entre ambas imágenes. Como se explicó anteriormente, queremos encontrar la transformación que asigna puntos desde el plano de la superficie al plano de la imagen. Esta transformación tendrá que actualizarse cada nuevo cuadro que procesemos.

En el desarrollo del proyecto de Realidad Aumentada, varios métodos matemáticos y algoritmos son fundamentales.





Las coordenadas  $u, v$  (coordenadas en el plano de la imagen) de un punto  $p$  expresadas en el sistema de coordenadas de la cámara si asumimos que una cámara estenopeica se pueden calcular como (la derivación de la ecuación se deja como ejercicio para el lector):

$$\begin{bmatrix} u \cdot k \\ v \cdot k \\ k \end{bmatrix} = \begin{bmatrix} f_u & 0 & u_0 \\ 0 & f_v & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x^{\text{cam}} \\ y^{\text{cam}} \\ z^{\text{cam}} \end{bmatrix}$$

**Calibration matrix**

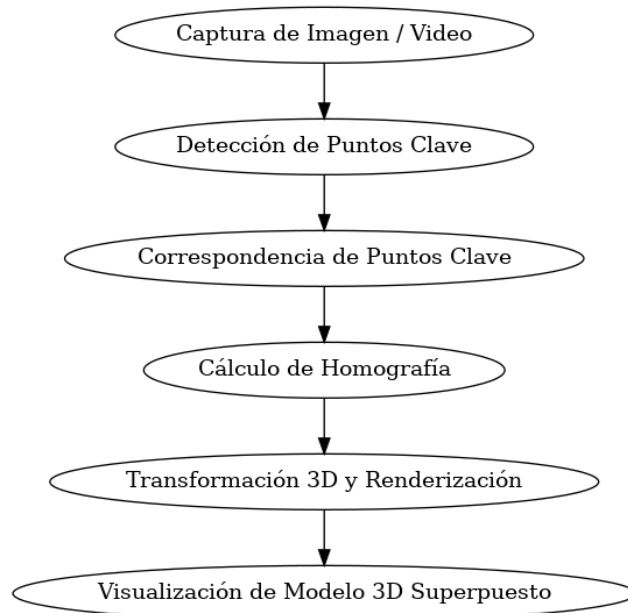
$(u_0, v_0)$  Projection of the optical center  
 $f_u, f_v$  Focal lengths

Donde la distancia focal es la distancia desde el orificio hasta el plano de la imagen, la proyección del centro óptico es la posición del centro óptico en el plano de la imagen y  $k$  es un factor de escala. La ecuación anterior nos dice entonces cómo se forma la imagen. Sin embargo, como se indicó anteriormente, conocemos las coordenadas del punto  $p$  en el sistema de coordenadas Mundial y no en el sistema de coordenadas de la Cámara, por lo que tenemos que agregar otra transformación que asigne puntos del sistema de coordenadas Mundial al sistema de coordenadas de la Cámara. La transformación que nos dice las coordenadas en el plano imagen de un punto  $p$  en el sistema de coordenadas Mundial es entonces:

$$\begin{bmatrix} u \cdot k \\ v \cdot k \\ k \end{bmatrix} = \underbrace{\begin{bmatrix} f_u & 0 & u_0 \\ 0 & f_v & v_0 \\ 0 & 0 & 1 \end{bmatrix}}_{\text{Calibration matrix}} \underbrace{\begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix}}_{\text{External calibration matrix}} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$$= \underbrace{\begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix}}_{\text{Projection matrix: 3x4 matrix}} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Ahora se explicará un lo que se requirió entender para realizar los códigos:



## 1. Detección de Puntos Clave y Descriptores (ORB)

- **ORB (Oriented FAST and Rotated BRIEF)**: Este algoritmo combina el detector de características FAST y el descriptor BRIEF, con algunas mejoras.
  - **FAST (Features from Accelerated Segment Test)**: Se usa para detectar esquinas y puntos clave en la imagen. El algoritmo considera un círculo de 16 píxeles

alrededor del punto de interés y, si un número suficiente de píxeles en este círculo es más claro u oscuro que el punto central, se considera una esquina.

- **BRIEF (Binary Robust Independent Elementary Features):** Genera un descriptor binario de los puntos clave detectados. Este proceso implica comparar intensidades de píxeles en pares en una región pequeña alrededor del punto clave.

## 2. Correspondencia de Puntos Clave

- **Algoritmo de Fuerza Bruta con Distancia de Hamming:** Se utiliza para encontrar las mejores coincidencias entre los descriptores de dos conjuntos de puntos clave. La distancia de Hamming mide el número de bits que difieren entre dos cadenas de bits (usado aquí debido a la naturaleza binaria de los descriptores BRIEF).

## 3. Cálculo de Homografía

- **Homografía:** Una homografía es una transformación que mapea los puntos de una imagen a otra manteniendo la perspectiva. En términos matemáticos, es una transformación proyectiva 2D que se puede representar como una matriz  $3 \times 3$ .
  - Se calcula usando el algoritmo RANSAC (RANdom SAmple Consensus) para encontrar la mejor homografía que alinea los puntos clave de la imagen objetivo con los puntos clave correspondientes en el fotograma del video.
  - Matemáticamente, si  $(p)$  es un punto en una imagen y  $(p')$  es el punto correspondiente en otra imagen, la homografía  $(H)$  satisface  $(p' = Hp)$ , donde  $(p)$  y  $(p')$  son representados en coordenadas homogéneas.

## 4. Transformación 3D y Proyección

- **Matriz de Proyección 3D:** Combinando la homografía calculada con los parámetros de la cámara, se genera una matriz de proyección 3D. Esta matriz se utiliza para mapear los puntos del modelo 3D en el espacio de la imagen del fotograma capturado.
  - Incluye cálculos para rotación, traslación y escalado, los cuales ajustan el modelo 3D a la perspectiva y escala correctas en el fotograma del video.

Este diagrama ayudará a comprender cómo se manejan y procesan las imágenes desde la captura inicial hasta la visualización final del modelo 3D superpuesto.

Los pasos representados en el diagrama son:

1. **Captura de Imagen / Video:** Inicio del proceso con la captura de imágenes o secuencias de video.
2. **Detección de Puntos Clave:** Procesamiento de las imágenes capturadas para identificar puntos clave significativos.

3. **Correspondencia de Puntos Clave:** Comparación y asociación de los puntos clave detectados en la imagen objetivo con los del video.
4. **Cálculo de Homografía:** Transformación de la perspectiva para alinear la imagen objetivo con la escena del video.
5. **Transformación 3D y Renderización:** Aplicación de la transformación 3D y renderización del modelo 3D en la ubicación adecuada dentro del fotograma de video.
6. **Visualización de Modelo 3D Superpuesto:** Presentación del resultado final donde el modelo 3D aparece integrado de manera realista en el entorno capturado por el video.

## Código

Resumen:

1. **Detección y Correspondencia de Características (Código 1):**
  - Utilizamos la biblioteca OpenCV para la captura de imágenes y el procesamiento de video.
  - El algoritmo ORB se emplea para detectar puntos clave y generar descriptores tanto en la imagen objetivo como en las capturas de video.
  - Se realizan coincidencias de puntos clave y se calcula una homografía para mapear la imagen objetivo dentro del video.
  - Se utiliza una proyección 3D para renderizar un modelo 3D sobre la superficie detectada.
2. **Carga y Manejo de Modelos 3D (Código 2):**
  - Se define una clase **OBJ** para cargar y procesar archivos de modelo 3D en formato OBJ.
  - La clase maneja vértices, normales, coordenadas de textura y caras del modelo 3D.

Vamos a desglosar y explicar detalladamente los pasos que llevan a cabo los dos códigos que se mostrarán a continuación.

### Código 1 - Proceso Principal de Realidad Aumentada

1. **Importación de Bibliotecas y Módulos:**

- Se importan las bibliotecas necesarias para el procesamiento de imágenes y gráficos 3D (OpenCV, Numpy, Math, OS) y un módulo personalizado para cargar modelos 3D.

## 2. Configuración Inicial:

- Se definen parámetros como el número mínimo de coincidencias requeridas para validar el reconocimiento y los parámetros predeterminados de color.

## 3. Función Principal main:

- Carga de la imagen de referencia (**model.jpg**) y del modelo 3D (**fox.obj**).
- Inicialización del detector ORB y el objeto BFMatcher para la correspondencia de puntos clave.
- Configuración de los parámetros de la cámara y la matriz de homografía.
- Se inicia la captura de video desde la cámara.

## 4. Bucle de Captura de Video:

- Captura continua de fotogramas del video.
- Detección de puntos clave y cálculo de descriptores para cada fotograma y para la imagen de referencia.
- Realización de coincidencias entre los descriptores del modelo y los del fotograma.

## 5. Validación y Procesamiento de Coincidencias:

- Si se encuentran suficientes coincidencias, se calcula la homografía.
- Con la homografía, se proyectan los puntos del modelo en el fotograma.
- Se renderiza el modelo 3D utilizando la matriz de proyección calculada.
- Se manejan las opciones para dibujar rectángulos, puntos clave y coincidencias según los argumentos del programa.

## 6. Visualización y Control de Salida:

- Se muestra el resultado en una ventana.
- Se proporciona un control para cerrar el programa presionando la tecla 'q'.

## 7. Limpieza:

- Al salir del bucle, se liberan los recursos de la cámara y se cierran todas las ventanas de OpenCV.



## ar\_main.py

```
import argparse

import cv2
import numpy as np
import math
import os
from objloader_simple import *

# Número mínimo de coincidencias que deben encontrarse
# para considerar que el reconocimiento es válido
MIN_MATCHES = 10
DEFAULT_COLOR = (0, 0, 0)

def main():
    """
    Esta función carga la imagen de la superficie objetivo,
    """
    homography = None
    # matriz de parámetros de la cámara (inventada pero funciona
    # bastante bien para mí)
    camera_parameters = np.array([[800, 0, 320], [0, 800, 240], [0,
0, 1]])
    # crear el detector de puntos clave ORB
    orb = cv2.ORB_create()
    # crear el objeto BFMatcher basado en la distancia de Hamming
    bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
    # cargar la superficie de referencia que se buscará en la
    # transmisión de video
    dir_name = os.getcwd()
    print("dir_name", dir_name)
    model = cv2.imread(os.path.join(dir_name, 'model.jpg'), 0)
    # Calcular los puntos clave y descriptores del modelo
    kp_model, des_model = orb.detectAndCompute(model, None)
    # Cargar el modelo 3D desde el archivo OBJ
    obj = OBJ(os.path.join(dir_name, 'fox.obj'), swapyz=True)
    # inicializar la captura de video
    cap = cv2.VideoCapture(0)

    while True:
        # leer el fotograma actual
        ret, frame = cap.read()
        if not ret:
            print("No se puede capturar el video")
```

```

        return

    # encontrar y dibujar los puntos clave del fotograma
    kp_frame, des_frame = orb.detectAndCompute(frame, None)
    # hacer coincidir los descriptores del fotograma con los
    descriptores del modelo
    matches = bf.match(des_model, des_frame)
    # ordenarlas en función de su distancia
    # cuanto menor sea la distancia, mejor será la coincidencia
    matches = sorted(matches, key=lambda x: x.distance)

    # calcular la homografía si se encuentran suficientes
    coincidencias
    if len(matches) > MIN_MATCHES:
        # diferenciar entre puntos de origen y puntos de destino
        src_pts = np.float32([kp_model[m.queryIdx].pt for m in
matches]).reshape(-1, 1, 2)
        dst_pts = np.float32([kp_frame[m.trainIdx].pt for m in
matches]).reshape(-1, 1, 2)
        # calcular la homografía
        homography, mask = cv2.findHomography(src_pts, dst_pts,
cv2.RANSAC, 5.0)
        if args.rectangle:
            # Dibujar un rectángulo que marca el modelo
            encontrado en el fotograma
            h, w = model.shape
            pts = np.float32([[0, 0], [0, h - 1], [w - 1, h -
1], [w - 1, 0]]).reshape(-1, 1, 2)
            # proyectar esquinas en el fotograma
            dst = cv2.perspectiveTransform(pts, homography)
            # conectarlas con líneas
            frame = cv2.polylines(frame, [np.int32(dst)], True,
255, 3, cv2.LINE_AA)
            # si se encontró una matriz de homografía válida,
            renderizar el cubo en el plano del modelo
            if homography is not None:
                try:
                    # obtener la matriz de proyección 3D a partir de
                    la matriz de homografía y los parámetros de la cámara
                    projection =
projection_matrix(camera_parameters, homography)
                    # proyectar el cubo o modelo
                    frame = render(frame, obj, projection, model,
False)

                    #frame = renderizar(frame, modelo, proyeccion)
                except:

```

```

        pass
        # dibujar las primeras 10 coincidencias.
        if args.matches:
            frame = cv2.drawMatches(model, kp_model, frame,
kp_frame, matches[:10], 0, flags=2)
            # mostrar el resultado

            cv2.imshow('frame', frame)
            if cv2.waitKey(1) & 0xFF == ord('q'):
                break

        else:
            print("No se encontraron suficientes coincidencias -
%d/%d" % (len(matches), MIN_MATCHES))

    cap.release()
    cv2.destroyAllWindows()
    return 0

def render(img, obj, projection, model, color=False):
    """
    Renderizar un modelo OBJ cargado en el fotograma de video actual
    """
    vertices = obj.vertices
    scale_matrix = np.eye(3) * 3
    h, w = model.shape

    for face in obj.faces:
        face_vertices = face[0]
        points = np.array([vertices[vertex - 1] for vertex in
face_vertices])
        points = np.dot(points, scale_matrix)
        # renderizar el modelo en el centro de la superficie de
referencia. Para hacerlo,
        # los puntos del modelo deben desplazarse
        points = np.array([[p[0] + w / 2, p[1] + h / 2, p[2]] for p
in points])
        dst = cv2.perspectiveTransform(points.reshape(-1, 1, 3),
projection)
        imgpts = np.int32(dst)
        if color is False:
            cv2.fillConvexPoly(img, imgpts, DEFAULT_COLOR)
        else:
            color = hex_to_rgb(face[-1])
            color = color[::-1] # invertir

```

```

        cv2.fillConvexPoly(img, imgpts, color)

    return img

def projection_matrix(camera_parameters, homography):
    """
    A partir de la matriz de calibración de la cámara y la
    homografía estimada,
    calcular la matriz de proyección 3D
    """
    # Calcular la rotación a lo largo de los ejes x e y, así como la
    traslación
    homography = homography * (-1)
    rot_and_transl = np.dot(np.linalg.inv(camera_parameters),
    homography)
    col_1 = rot_and_transl[:, 0]
    col_2 = rot_and_transl[:, 1]
    col_3 = rot_and_transl[:, 2]
    # normalizar vectores
    l = math.sqrt(np.linalg.norm(col_1, 2) * np.linalg.norm(col_2,
    2))
    rot_1 = col_1 / l
    rot_2 = col_2 / l
    translation = col_3 / l
    # calcular la base ortonormal
    c = rot_1 + rot_2
    p = np.cross(rot_1, rot_2)
    d = np.cross(c, p)
    rot_1 = np.dot(c / np.linalg.norm(c, 2) + d / np.linalg.norm(d,
    2), 1 / math.sqrt(2))
    rot_2 = np.dot(c / np.linalg.norm(c, 2) - d / np.linalg.norm(d,
    2), 1 / math.sqrt(2))
    rot_3 = np.cross(rot_1, rot_2)
    # finalmente, calcular la matriz de proyección 3D del modelo al
    fotograma actual
    projection = np.stack((rot_1, rot_2, rot_3, translation)).T
    return np.dot(camera_parameters, projection)

def hex_to_rgb(hex_color):
    """
    Helper function to convert hex strings to RGB
    """
    hex_color = hex_color.lstrip('#')
    h_len = len(hex_color)

```

```

        return tuple(int(hex_color[i:i + h_len // 3], 16) for i in
range(0, h_len, h_len // 3))

# Análisis de argumentos de línea de comandos
# NO TODOS SON SOPORTADOS AÚN
parser = argparse.ArgumentParser(description='Aplicación de realidad
aumentada')

parser.add_argument('-r', '--rectangle', help = 'dibujar un
rectángulo delimitador de la superficie objetivo en el fotograma',
action = 'store_true')
parser.add_argument('-mk', '--model_keypoints', help = 'dibujar los
puntos clave del modelo', action = 'store_true')
parser.add_argument('-fk', '--frame_keypoints', help = 'dibujar los
puntos clave del fotograma', action = 'store_true')
parser.add_argument('-ma', '--matches', help = 'dibujar coincidencias
entre puntos clave', action = 'store_true')
# TODO jgallostraa -> agregar soporte para especificar el modelo
#parser.add_argument('-mo', '--model', help = 'Especificar el modelo
a proyectar', action = 'store_true')
args = parser.parse_args()

if __name__ == '__main__':
    main()

```

## Código 2 - Carga de Modelos 3D

### 1. Definición de la Clase OBJ:

- Esta clase se encarga de cargar y procesar un archivo modelo 3D en formato **.obj**.

### 2. Inicialización y Lectura del Archivo:

- En el constructor de la clase, se inicializan listas para almacenar vértices, normales, coordenadas de textura y caras.
- Se lee el archivo **.obj** línea por línea y se procesan las distintas partes del modelo.

### 3. Procesamiento de Vértices y Otros Elementos:

- Se identifican y almacenan los vértices (**v**), normales (**vn**), y coordenadas de textura (**vt**) en sus respectivas listas.
- Se procesan las caras (**f**), que pueden incluir referencias a vértices, normales y coordenadas de textura.

### 4. Opciones de Transformación:

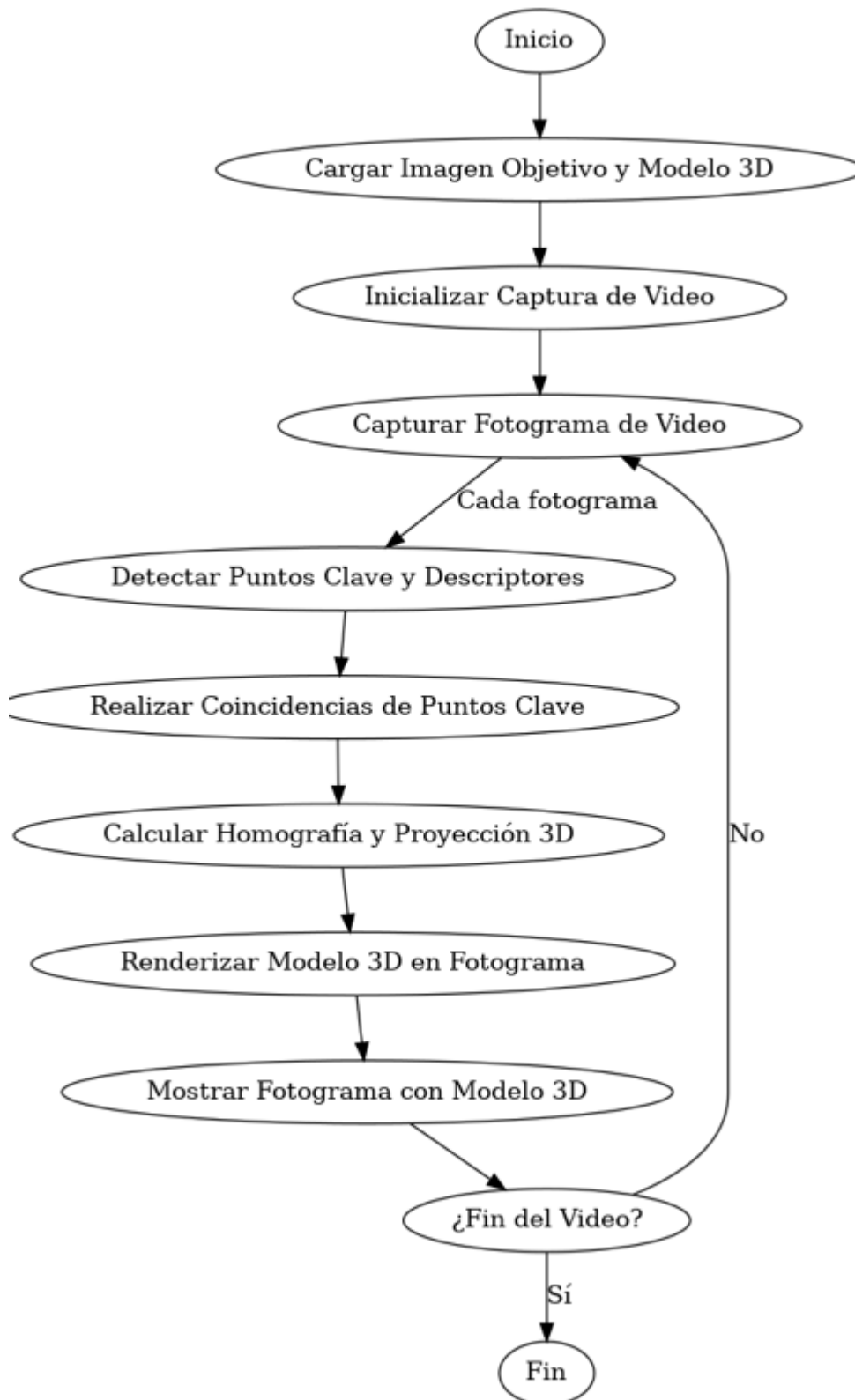
- Se proporciona la opción **swapyz** para intercambiar las coordenadas y y z durante la carga, lo cual puede ser necesario dependiendo de la convención de coordenadas del archivo **.obj**.

#### objloader\_simple.py

```
class OBJ:
    def __init__(self, filename, swapyz=False):
        """Carga un archivo Wavefront OBJ."""
        self.vertices = []      # Lista para almacenar vértices
        self.normals = []       # Lista para almacenar normales
        self.texcoords = []     # Lista para almacenar coordenadas de
        textura
        self.faces = []         # Lista para almacenar caras
        material = None
        for line in open(filename, "r"):
            if line.startswith('#'): continue
            values = line.split()
            if not values: continue
            if values[0] == 'v':
                v = list(map(float, values[1:4]))
                if swapyz:
                    v = v[0], v[2], v[1]
                self.vertices.append(v)
            elif values[0] == 'vn':
                v = list(map(float, values[1:4]))
                if swapyz:
                    v = v[0], v[2], v[1]
                self.normals.append(v)
            elif values[0] == 'vt':
                self.texcoords.append(map(float, values[1:3]))
            #elif values[0] in ('usemtl', 'usemat'):
            #    material = values[1]
            #elif values[0] == 'mtllib':
            #    self.mtl = MTL(values[1])
            elif values[0] == 'f':
                face = []
                texcoords = []
                norms = []
                for v in values[1:]:
                    w = v.split('/')
                    face.append(int(w[0]))
                    if len(w) >= 2 and len(w[1]) > 0:
                        texcoords.append(int(w[1]))
```

```
        else:
            texcoords.append(0)
            if len(w) >= 3 and len(w[2]) > 0:
                norms.append(int(w[2]))
            else:
                norms.append(0)
        #self.faces.append((face, norms, texcoords,
material))
        self.faces.append((face, norms, texcoords))
```

## Explicación





El diagrama de flujo que he creado representa el proceso del sistema de Realidad Aumentada, basado en los códigos que se mostraron.

**1. Inicio (A):**

- Este es el punto de partida del programa. Aquí se inician todas las configuraciones necesarias.

**2. Cargar Imagen Objetivo y Modelo 3D (B):**

- Se cargan dos elementos clave: la imagen objetivo que se reconocerá en el video y el modelo 3D que se superpondrá sobre ella.
- La imagen objetivo se utiliza para detectar dónde colocar el modelo 3D.
- El modelo 3D es el objeto que se renderizará en la posición detectada.

**3. Inicializar Captura de Video (C):**

- Se inicia la captura de video, generalmente desde una cámara web, para obtener los fotogramas que serán procesados.

**4. Capturar Fotograma de Video (D):**

- En este paso, se captura un fotograma individual del flujo de video.
- Este proceso se repite para cada nuevo fotograma del video.

**5. Detectar Puntos Clave y Descriptores (E):**

- Se detectan puntos clave tanto en la imagen objetivo como en el fotograma capturado.
- Se generan descriptores que ayudan a identificar y comparar estos puntos clave entre la imagen objetivo y el fotograma.

**6. Realizar Coincidencias de Puntos Clave (F):**

- Los descriptores obtenidos del fotograma se comparan con los de la imagen objetivo para encontrar coincidencias.
- Este proceso ayuda a identificar la posición y orientación de la imagen objetivo dentro del fotograma del video.

**7. Calcular Homografía y Proyección 3D (G):**

- Con las coincidencias encontradas, se calcula una homografía. Esta es una transformación que permite mapear la imagen objetivo al fotograma.
- También se calcula la proyección 3D que se usará para posicionar correctamente el modelo 3D en relación con la cámara.

**8. Renderizar Modelo 3D en Fotograma (H):**

- Una vez calculada la homografía y la proyección, el modelo 3D se renderiza o dibuja sobre el fotograma del video en la posición y orientación correctas.

**9. Mostrar Fotograma con Modelo 3D (I):**

- El fotograma procesado, con el modelo 3D superpuesto, se muestra en la pantalla.

**10. ¿Fin del Video? (J):**

- Aquí se verifica si el video ha terminado o si el usuario ha decidido terminar el proceso (por ejemplo, presionando una tecla específica).
- Si el video no ha terminado, el proceso vuelve al paso de "Capturar Fotograma de Video" (D).
- Si el video ha terminado, se procede al siguiente paso.

**11. Fin (K):**

- Este es el final del proceso. Aquí se liberan los recursos utilizados, como la cámara, y se cierran todas las operaciones.

Resultados:



## Conclusión:

Este proyecto de Realidad Aumentada (RA) demuestra una aplicación sofisticada de técnicas de visión por computadora y gráficos 3D, utilizando Python y OpenCV. A través de la implementación y análisis de los códigos proporcionados, se ha logrado una comprensión profunda de varios conceptos y metodologías clave en el campo de la RA.

Primero, la utilización del algoritmo ORB para la detección de puntos clave y la generación de descriptores ha demostrado ser una técnica eficiente y robusta para el reconocimiento de patrones en imágenes. Esto es crucial en RA, donde la precisión y rapidez en la detección de la imagen objetivo son fundamentales para una experiencia de usuario fluida.

En segundo lugar, la aplicación de homografías para mapear una imagen 2D en un espacio 3D y la posterior renderización de modelos 3D en tiempo real ilustra el poder de las transformaciones geométricas en la creación de experiencias inmersivas. Esto no solo refuerza la comprensión de la geometría y la proyección en el espacio 3D, sino que también destaca la importancia de una adecuada calibración y ajuste de los parámetros de la cámara.

Además, el proyecto ha proporcionado una valiosa experiencia práctica en la manipulación y procesamiento de archivos de modelo 3D. La capacidad de cargar y procesar estos modelos es esencial en la RA y en muchas otras aplicaciones de gráficos computacionales.

## Fuentes Bibliograficas

[https://es.wikipedia.org/wiki/Realidad\\_aumentada](https://es.wikipedia.org/wiki/Realidad_aumentada)

<https://blog.hubspot.es/service/ejemplos-realidad-aumentada>

<https://codigospython.com/python-ar-realidad-aumentada-con-python/>

[https://www.w3schools.com/python/ref\\_func\\_open.asp](https://www.w3schools.com/python/ref_func_open.asp)

<https://medium.com/un-enfoque-cient%C3%ADfico-de-python/opencv-con-python-2a9b582e22cd>